Data Types and Tokens

Contents

- Identifiers
- Data Types
- Operators
- Special Operator
- Local and Global Variables
- Tokens

Identifiers

- In C++, an identifier is a name used to identify variables, functions, classes, objects, and other programming entities. Identifiers are an essential part of the language and are used to give *meaningful names* to various elements in your code.
- Here are some rules and conventions for naming identifiers in C++:
  - Identifiers must start with a letter (uppercase or lowercase) or an underscore _.
  - The subsequent characters can be letters, digits, or underscores.
  - Identifiers are case-sensitive, meaning *myVar* and *myvar* are treated as different identifiers.
  - C++ reserves some keywords that cannot be used as identifiers, like int, class, if, while, etc.
  - Identifiers should not use special characters like !, @, $, etc.

Data Types

Here’s a list of common C++ data types along with their typical sizes in bytes. Keep in mind that these sizes can vary depending on the platform, compiler, and architecture you're using.

- Integral Data Types:
  - **bool:** Typically 1 byte
  - **char:** Typically 1 byte
  - **int:** Typically 4 bytes
  - **long or long int:** Typically 4 bytes (32-bit), 8 bytes (64-bit)
  - **long long or long long int:** Typically 8 bytes
- Floating-Point Data Types:
  - **float:** Typically 4 bytes
  - **double:** Typically 8 bytes
  - **long double:** Typically 8 or 16 bytes
- Derived Data Types:
  - **pointer:** Typically 4 bytes (32-bit), 8 bytes (64-bit)
  - **array:** Size depends on the size of the elements and the number of elements
- User-Defined Data Types:
  - **class:** Size depends on the size of its member variables, along with any overhead for virtual functions and inheritance.

You can use the *sizeof* operator to determine the size of a particular data type on your specific system.

Operators

- Arithmetic Operator (+, -, *, / , %)
- Assignment Operator (=, +=, -=, *=, /=)
- Relational Operator (==, !=, >, <, >=, <=)
- Logical Operators (&&, ||, |)
- Special Operators

Special Operator

- These special operators are used for specific purposes and often involve more advanced concepts. Here are some of the notable special operators in C++:
- Member Access Operators:
  - . (Dot Operator): Used to access a member of an object directly.
  - -> (Arrow Operator): Used to access a member of a pointer to an object.
- Sizeof Operator:
  - sizeof: Returns the size (in bytes) of a data type or an object.
- Conditional Operator:
  - ? : (Ternary Operator): Offers a concise way to express conditional statements.

Special Operator

- Pointer and Reference Operators:
  - * (Pointer Dereference Operator): Used to access the value pointed to by a pointer.
  - & (Address-of Operator): Used to get the address of a variable.
- Increment and Decrement Operators:
  - ++ (Increment Operator): Increases the value of a variable by one.
  - -- (Decrement Operator): Decreases the value of a variable by one.
- New and Delete Operators:
  - new: Allocates memory for objects dynamically.
  - delete: Deallocates memory allocated using new.
- Scope Resolution Operator:
  - :: (Scope Resolution Operator): Used to access members in a specific namespace, class, or global scope.

Local and Global Variables

- Local Variables:
  - **Scope:** Local variables are declared within a specific block of code, such as a function or a block within a function. They are only accessible within that block and its nested blocks.
  - **Lifetime:** The lifetime of a local variable begins when the block containing the variable is entered and ends when the block is exited.
- Global Variables:
  - **Scope:** Global variables are declared outside of any function, typically at the top of the file. They are accessible from anywhere in the program, including all functions.
  - **Lifetime:** The lifetime of a global variable is the entire duration of the program.

Tokens

- In C++, a token is the smallest meaningful unit in the source code of a program.
- **Keywords:** Keywords are reserved words that have a specific meaning in the C++ language and cannot be used as identifiers (variable or function names). Examples include int, if, while, class, return, etc.
- **Identifiers:** Identifiers are user-defined names used for variables, functions, classes, and other programming entities. An identifier must start with a letter or an underscore and can be followed by letters, digits, or underscores.
- **Literals:** Literals represent constant values that are directly used in the program. There are several types of literals:
  - Integer literals (e.g., 42, -123)
  - Floating-point literals (e.g., 3.14, -0.01)
  - Character literals (e.g., 'A', '9', '%')
  - String literals (e.g., "Hello, world!")
  - Boolean literals (true and false)

Tokens

- **Operators:** Operators perform operations on operands. C++ has various types of operators, such as arithmetic operators (+, -, *, /), relational operators (<, >, <=, >=, ==, !=), logical operators (&&, ||, !), etc.
- **Punctuation Symbols:** These symbols are used to structure the code and include:
  - Braces { and } for defining code blocks.
  - Parentheses (and) for function calls and expressions.
  - Square brackets [and] for arrays and index access.
  - Semicolon ; to terminate statements.
  - Comma , to separate items in lists.

Tokens

- **Comments:** Comments are used to provide explanations or notes within the code. They are not interpreted by the compiler and do not affect the program's behavior.
  - Single-line comment: // This is a single-line comment
  - Multi-line comment: /* This is a multi-line comment */
- **Preprocessor Directives:** Preprocessor directives are instructions to the preprocessor, which processes the code before actual compilation. They start with a # symbol. Examples include #include, #define, #ifdef, etc.
- **Whitespace:** Whitespace includes spaces, tabs, and newline characters. It separates tokens and helps improve code readability.